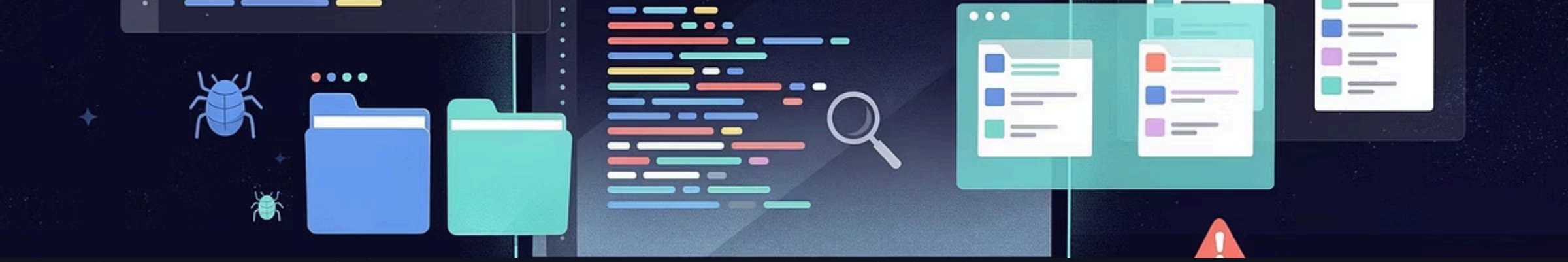

Progetti Pratici — Giorno 7: Eccezioni e File I/O

L'obiettivo non è evitare gli errori, ma imparare a gestirli, trasformarli in messaggi chiari e mantenere il programma controllato.

Progetto 1: Config Guardian

Lettura e validazione di un file di configurazione con gestione separata di errori tecnici e di dominio.

Progetto 2: Order Import System

Lettura di un ordine da file, validazione, calcolo totale e scrittura di un report.

I due progetti coprono complessivamente circa 4 ore di pratica autonoma guidata.

Timing delle 4 ore

Il docente guida l'avvio di ogni progetto, poi gli studenti lavorano in autonomia progressiva.

20 min — Introduzione e setup file

Lettura della consegna, creazione struttura progetto, analisi del file di input.

80 min — Progetto 1: Config Guardian

Implementazione guidata con autonomia crescente.

20 min — Revisione collettiva Progetto 1

Confronto soluzioni, errori comuni, domande.

110 min — Progetto 2: Order Import System

Progetto più complesso, lavoro autonomo.

10 min — Recap errori comuni

Consolidamento dei pattern corretti.

Totale: 240 minuti — 4 ore di pratica

Progetto 1 — Config Guardian

Scenario

L'applicazione legge un file `config.txt` contenente le impostazioni base dell'applicazione.

Il file contiene quattro campi:

- `nomeApplicazione`
- `versione`
- `ambiente`
- `porta`

> Questo scenario è pensato per esercitarsi nella lettura di file e nella gestione delle eccezioni legate alla loro integrità e disponibilità.

80 MINUTI

Obiettivo

L'obiettivo è distinguere chiaramente due tipi di errori:

- `IOException` — errore tecnico: il file non esiste o non è leggibile.
- `ConfigurazioneNonValidaException` — errore logico: il file esiste ma i dati non rispettano le regole.

> Questi due tipi di errore hanno cause diverse, messaggi diversi e devono essere gestiti separatamente.

Config Guardian — Classi richieste

ConfigurazioneApp

Rappresenta i dati della configurazione: nomeApplicazione, versione, ambiente, porta. Usa un record o una classe con costruttore e metodi di accesso.

ConfigurazioneReader

Legge il file config.txt usando Path e Files.readString. Lancia IOException se il file non è accessibile. Non contiene logica di validazione.

ConfigurazioneValidator

Controlla che i dati rispettino le regole di dominio. Lancia ConfigurazioneNonValidaException se qualcosa non va. Non legge file.

ConfigurazioneNonValidaException

Custom exception che estende RuntimeException. Contiene un messaggio chiaro che descrive quale campo non è valido e perché.

Main

Coordina il flusso: chiama Reader, poi Validator, poi stampa il risultato. Gestisce le eccezioni con messaggi distinti per errore tecnico e logico.

Ogni classe ha una sola responsabilità. Il Main non contiene logica di lettura né di validazione.

Config Guardian — Regole e criteri di accettazione

📋 Regole di validazione

01

nomeApplicazione non deve essere vuoto o blank.

02

versione non deve essere vuota o blank.

03

ambiente deve essere esattamente uno tra: DEV, TEST o PROD.

04

porta deve essere un numero intero compreso tra 1024 e 65535.

05

Se il file non esiste o non è leggibile, gestire come IOException — non come errore di validazione.

> Se il programma stampa solo 'errore', non ha ancora finito il lavoro.

✅ Criteri di accettazione

- Il programma non crasha mai silenziosamente.
- IOException e ConfigurazioneNonValidaException producono messaggi distinti e chiari.
- Il Main coordina ma non contiene logica di lettura o validazione.
- I messaggi di errore indicano quale campo è non valido e perché.

Progetto 2 — Order Import System

Scenario

L'applicazione legge un ordine da un file `ordine.txt`, lo trasforma in un oggetto Java, lo valida e genera un report finale scritto su `report-ordine.txt`.

Il file contiene quattro campi:

- `cliente`
- `prodotto`
- `quantita`
- `prezzoUnitario`

> Questo scenario è pensato per integrare diversi concetti: lettura e scrittura di file, gestione delle eccezioni personalizzate e validazione dei dati.

110 MINUTI

Obiettivo

Integrare in un unico flusso di lavoro:

- File I/O con Path e Files
- Custom exception per errori di dominio
- Parsing base di stringhe in numeri
- Validazione con regole chiare
- Incapsulamento e separazione delle responsabilità
- Scrittura del report sia in caso di successo sia in caso di errore

> Il programma deve funzionare anche quando i dati sono sbagliati — e deve dirlo chiaramente.

Progetto 2 — Order Import System — Classi richieste

Ordine

Contiene: cliente (String), prodotto (String), quantita (int), prezzoUnitario (double). Espone un metodo calcolaTotale() e un metodo riepilogo() che restituisce una stringa formattata con tutti i dati dell'ordine.

OrdineReader

Legge il file ordine.txt con Path e Files.readString. Estrae i quattro campi e restituisce un oggetto Ordine grezzo. Lancia IOException se il file non è accessibile.

OrdineValidator

Valida le regole dell'ordine. Lancia OrdineNonValidoException se i dati non rispettano i vincoli. Non legge file e non scrive report.

OrdineNonValidoException

Custom exception che estende RuntimeException. Il messaggio deve indicare quale campo non è valido e il motivo.

ReportWriter

Scrive il file report-ordine.txt con il risultato dell'elaborazione. Deve essere chiamato sia in caso di successo sia in caso di errore. Lancia IOException se la scrittura fallisce.

Main

Coordina il flusso: legge, valida, scrive il report. Gestisce IOException e OrdineNonValidoException con messaggi separati e chiari.

Ogni classe ha una sola responsabilità. Il Main non contiene logica di parsing, validazione o scrittura.

Order Import System — Regole di validazione

📋 Regole sui dati

01

cliente non deve essere vuoto o blank.

02

prodotto non deve essere vuoto o blank.

03

quantita deve essere un intero maggiore di 0.

04

prezzoUnitario deve essere un numero decimale maggiore di 0.

05

Se quantita o prezzoUnitario non sono numerici, il programma deve generare un messaggio chiaro — non uno stack trace grezzo.

06

Se il file ordine.txt non esiste, gestire come IOException.

📄 Regole sul report

Il file report-ordine.txt deve essere scritto sempre: sia in caso di successo sia in caso di errore.

In caso di successo, il report contiene il riepilogo completo dell'ordine con il totale calcolato.

In caso di errore, il report contiene il motivo del fallimento con un messaggio comprensibile.

> Il report è la prova che il programma ha lavorato — anche quando qualcosa è andato storto.

Output attesi — Order Import System

Il programma non deve mai stampare stack trace grezzi all'utente finale. I messaggi devono essere chiari, leggibili e informativi.

✅ Caso di successo

```
ORDINE IMPORTATO CON SUCCESSO
----------------------------
Cliente:        Mario Rossi
Prodotto:       Laptop Pro 15
Quantità:       2
Prezzo unitario:  € 899,00
Totale:         € 1.798,00
----------------------------
Report scritto in: report-ordine.txt
```

❌ Caso di errore

```
IMPORTAZIONE FALLITA
----------------------------
Motivo: [campo] non valido — [descrizione del problema]
oppure
Motivo: File ordine.txt non trovato
----------------------------
Report di errore scritto in: report-ordine.txt
```

Il formato esatto è indicativo. L'importante è che le informazioni siano presenti e leggibili.

Checklist finale di valutazione

Usa questa checklist per verificare il tuo lavoro prima della revisione collettiva. Ogni punto deve essere soddisfatto da entrambi i progetti.

Struttura e compilazione

- Il codice compila senza errori
- Ogni classe ha una sola responsabilità chiara
- Il Main coordina ma non contiene logica di lettura, validazione o scrittura
- I nomi delle classi e dei metodi sono descrittivi

Gestione errori

- IOException è gestita con un messaggio chiaro
- Le custom exception hanno nomi espressivi e messaggi informativi
- I catch non sono mai vuoti
- Exception generica non viene catturata senza motivo
- Errore tecnico e errore di dominio producono messaggi distinti

File e output

- Il file di input viene letto correttamente
- Il report viene scritto correttamente
- Il report viene scritto sia in caso di successo sia in caso di errore
- Gli errori sono comunicati con messaggi comprensibili all'utente
- Nessuno stack trace grezzo viene stampato all'utente finale

Errori comuni da evitare

Questi sono gli errori più frequenti nei progetti di oggi. Riconoscerli prima di commetterli è già metà del lavoro.

❌ Tutto nel Main

Mettere lettura, validazione, parsing e scrittura tutti nel metodo main. Il Main deve solo coordinare.

❌ Catch vuoti

Scrivere catch(Exception e) {} senza fare nulla. L'errore viene inghiottito e il programma sembra funzionare.

❌ Catturare Exception senza motivo

Usare catch(Exception e) quando si sa già che tipo di eccezione aspettarsi. Essere specifici.

❌ Messaggi generici

Stampare solo "errore" o "qualcosa è andato storto". Il messaggio deve dire cosa, dove e perché.

❌ Confondere file mancante con dato non valido

IOException e custom exception hanno cause diverse. Non gestirle con lo stesso catch.

❌ Non validare i dati letti da file

Assumere che il file contenga sempre dati corretti. I file possono contenere qualsiasi cosa.

❌ Report solo nel caso felice

Scrivere il report solo quando tutto va bene. Il report deve essere scritto sempre.

❌ Stack trace grezzo all'utente

Stampare e.printStackTrace() come output finale. L'utente non deve vedere stack trace tecnici.

Recap didattico — Giorno 7

Un programma professionale non presume che tutto vada bene. Verifica, gestisce e comunica.

Legge — usa Path e Files per leggere file in modo moderno e sicuro.

Valida — controlla i dati con regole chiare prima di usarli.

Gestisce — distingue errori tecnici da errori di dominio con eccezioni specifiche.

Comunica — produce messaggi chiari per l'utente, non stack trace grezzi.

> **Nel Giorno 7 l'errore smette di essere panico e diventa progettazione.**